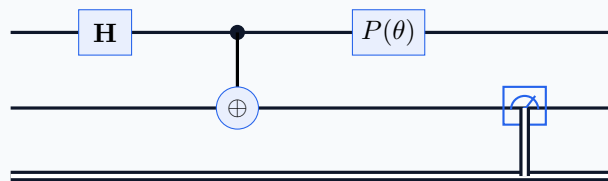


QPU Getting Started

Programming and Website Walkthrough



Bailey Forbes

September 13, 2026

A beginner-first introduction to the QPU website, programming vocabulary, a six-step learning path, classical logic, complete runnable protocols, a reference for every on-screen control, and the first steps for correcting an error.

Contents

1	How to use the QPU website	1
1.1	Five programming words used in this guide	1
1.2	Interface words and how they relate	1
1.3	Run a bundled example without writing code	1
1.4	Compile a complete protocol	1
1.5	Complete examples versus fragments	2
1.6	When protocol checks report a problem	2
1.7	Saving and reopening work	2
2	Learning path: six steps from one qubit to multi-stage circuits	2
2.1	Step 1: one qubit at a time	3
2.2	Step 2: controlled gates	3
2.3	Step 3: reversible Boolean logic	3
2.4	Step 4: entanglement and phase	3
2.5	Step 5: arithmetic	3
2.6	Step 6: multi-stage circuits	3
3	First circuit: from source to result	4
3.1	Read the program line by line	4
3.2	What one run means	4
3.3	Adding user-controlled input	4
4	Classical computation foundations	5
4.1	Classical bits and logical state	5
4.2	Boolean algebra	5
4.3	Truth tables as complete finite specifications	5
4.4	Combinational logic and stored state	6
4.5	Why ordinary classical gates can lose information	6
4.6	Controls, targets, and workspace	6
4.7	Fan-out, copying, and the no-cloning boundary	7
4.8	Deterministic, probabilistic, and quantum models	7
4.9	Classical and QPU operation map	7
5	Worked example: reversible half-adder	7
5.1	Classical specification	8
5.2	QPU protocol	8
5.3	What every term is doing	8
5.4	Build the Carry gate in the workbench	9
5.5	Reversible state equation	9
5.6	Companion truth-table metadata	9
6	Worked example: randomness versus interference	9

6.1	Classical fair coin model	10
6.2	Quantum preparation with Hadamard	10
6.3	Insert phase and recombine paths	10
6.4	Term-by-term explanation	10
6.5	Comparison table	11
7	Every control on screen, by its exact name	11
7.1	Navigation menu	11
7.2	Documentation in the Interactive workbench	11
7.3	Gate palette: “Pick up a block”	12
7.4	Wrapper dialog: “Gate wrappers”	13
7.5	Gate engine: “Register custom gates from QPU processes”	14
7.6	Circuit canvas: “Standard circuit diagram”	14
7.7	Interactive workbench: “Add particles, gates, and measurements”	15
7.8	Run controls	16
7.9	Examples: “Load a starter circuit”	16
7.10	QPU protocol compiler: “Compile text into a circuit”	16
7.11	Particle visualization	17
7.12	File upload and download	17
7.13	Circuit correction lab	17
8	Reset: an operation that is not a gate	17
8.1	Reversible gates versus RESET	18
8.2	Where RESET comes from	18
8.3	Reset buttons compared	18
9	Where to continue	19

1 How to use the QPU website

This chapter assumes no programming experience. You can build circuits by clicking controls, or edit the text protocol directly. Both paths describe the same circuit.

1.1 Five programming words used in this guide

Source text The human-readable instructions in the protocol editor. It is ordinary text, like a note, with one QPU instruction per logical line.

Instruction One action for the QPU system, such as SET, H, CNOT, or MEASURE. The first word says what action to perform.

Argument Extra information after an instruction. In SET Q 0p, Q and 0p are arguments: they say which wire and which state.

Compile Translate source text into the ordered visual gates that the simulator can run. Compilation checks spelling, required inputs, and targets.

Run Apply the compiled gates to the selected starting state. Running changes simulator state; it does not rewrite the source text.

1.2 Interface words and how they relate

Word on screen or in source	Meaning
Particle / qubit	One simulated quantum two-state system. The interface may say particle where circuit theory says qubit.
Wire	The horizontal visual path followed by one qubit through the circuit.
Token	A source-text name, such as Q or Carry, that resolves to a wire.
Register	One or more qubit wires considered as a group.
Gate	A box or controlled symbol that transforms state.
Target	The wire changed by a gate.
Control	A wire whose state decides whether a controlled gate acts.
Protocol / process	The complete source description of a named circuit.

1.3 Run a bundled example without writing code

1. Open the navigation menu and choose **Circuit builder**.
2. Find **Load a starter circuit** and choose an example.
3. Inspect the horizontal wires and gate blocks on the circuit canvas.
4. Choose **Play Sequence** to watch gates advance, or **Run all** to jump to the final state.
5. Choose **Measure all** if the circuit has not already measured its outputs.
6. Read the particle/state display and runtime log. A measured qubit is a classical 0 or 1 outcome for that run.
7. Choose **Reset state** before repeating the same experiment.

1.4 Compile a complete protocol

Listings marked **Complete runnable protocol** may be copied as a whole:

Complete runnable protocol

```
MAIN-PROCESS FirstWebsiteCircuit
CREATETOKEN -I Q
SET Q 0p
H -I Q -O Q
MEASURE -I Q
RETURNVALS Q
```

1. Scroll to **Compile text into a circuit**.
2. Replace the editor contents with the complete listing.
3. Choose **Compile protocol**.
4. Read **Protocol checks**. Zero errors means compilation may proceed. Warnings identify accepted syntax that may not behave as expected.
5. Return to the run controls and choose **Run all**.

Expected result: the H gate prepares equal 0 and 1 amplitudes. MEASURE returns either 0 or 1. Repeating from Reset state should produce both outcomes over many runs.

1.5 Complete examples versus fragments

This documentation uses three listing labels:

Complete runnable protocol Copy the entire block into the protocol editor and compile it.

Instruction fragment A few lines demonstrating one syntax rule. Add them to a complete process rather than compiling the fragment alone.

Companion metadata Truth-table text saved separately as `.qpuio`; do not paste it into the protocol editor.

1.6 When protocol checks report a problem

An **error** blocks successful compilation. A **warning** means the text is accepted but may use inactive, incomplete, or surprising behavior. Each diagnostic gives a line number, the relevant source line when available, and a suggested correction.

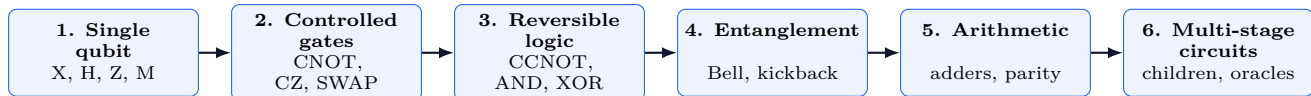
Choose **Open Correction Lab for guided help** below the diagnostic list when the correction is not obvious. The Correction Lab can test a protocol, probe outputs, explain failures, and propose gate-level guidance. It does not change protected bundled truth tables.

1.7 Saving and reopening work

Choose **Download as .qpucir** to save protocol source. The `-qpucir.txt` option contains the same source for devices that only handle plain text files. Use the navigation menu's **Upload files** page to reopen a protocol, optionally together with its matching `.qpuio` truth-table metadata.

2 Learning path: six steps from one qubit to multi-stage circuits

Each step below uses only ideas from the steps before it. You do not need to finish the theory chapters before starting: build the starter circuit first, predict what it should do, then read the matching explanation.



How to use each step

1. Open **Circuit builder** and choose the starter circuit named in the step from **Load a starter circuit**. Each card shows its step number.
2. Before running it, write down what you expect to measure.
3. Choose **Step gate** repeatedly and watch the highlighted column on the canvas and the **Execution log**.
4. Compare your prediction with the result, then read the listed guide section.
5. Move on when you can answer the step's checkpoint question without looking.

2.1 Step 1: one qubit at a time

Learn: a qubit starts in $|0\rangle$; X flips it; H creates an equal superposition; Z changes phase without changing immediate probabilities; MEASURE (the meter on the qubit wire) samples a classical 0 or 1 and marks that time on the **c** lane. The qubit can still be used afterward.

Starter circuits: *Superposition coin*, then *Phase interference (H-Z-H)*.

Read: *First circuit* and *Randomness versus interference* in Getting Started; *Single-qubit primitive gates* in the Theory Guide.

Checkpoint: why does H followed by M give 0 or 1 at random, while H, Z, H followed by M always gives 1?

2.2 Step 2: controlled gates

Learn: a controlled gate reads one wire (the *control*) and changes another (the *target*). The control is never modified by CNOT. In the builder, the control comes from **Control A** and the target from **Target particle**.

Starter circuit: *CNOT demo*. Change **q0 start** between **0p** and **1p** and rerun.

Read: *Controlled and two-qubit primitive gates* in the Theory Guide.

Checkpoint: in *CNOT demo*, which wire can change, and under what condition?

2.3 Step 3: reversible Boolean logic

Learn: quantum gates cannot throw information away, so reversible logic keeps its inputs and writes the answer into a separate *target* (also called workspace). Toffoli (CCNOT) and the derived AND gate both compute $t' = t \oplus (A \wedge B)$: A and B are read and left unchanged, and t is the output wire. Start t at 0 and it ends holding $A \wedge B$.

Starter circuits: *CCNOT demo*, then *Half adder*.

Read: *Classical computation foundations* and *Worked example: reversible half-adder* in Getting Started; *Derived logical gates* in the Theory Guide.

Checkpoint: if the target of AND starts at 1 instead of 0, what does it hold afterward?

2.4 Step 4: entanglement and phase

Learn: a controlled gate whose control is in superposition can correlate two wires so that neither has a separate state (a Bell state). Phase can also flow *backward* from target to control (phase kickback).

Starter circuits: *Bell state*, then *Phase kickback*.

Read: *Multiple qubits* (Entanglement) and *Measurement, probability, and truth tables* in the Theory Guide; *Phase kickback* in Examples and Troubleshooting.

Checkpoint: after *Bell state*, if q0 measures 1, what will q1 measure? Why is that not the same as copying q0?

2.5 Step 5: arithmetic

Learn: addition is built from the step 3 pieces. Sum bits are parity (XOR), carry bits are majority (a combination of ANDs). A comparator or parity checker uses the same pieces with a different output.

Starter circuits: *Parity check*; then compile the bundled **SingleBitFullAdder** and **TwoBitFullAdder** buttons in **Compile text into a circuit**.

Read: *Advanced worked circuits* in Examples and Troubleshooting (full adder, two-bit adder, comparators, parity).

Checkpoint: why does a full adder need three AND-style gates for its carry, but only XOR-style gates for its sum?

2.6 Step 6: multi-stage circuits

Learn: larger circuits are built in stages: compute a helper value, use it, then *uncompute* it so workspace returns to 0. Child processes (RUNCHILD) and custom gates reuse whole circuits as single steps. A recursive child (REC/TREC with

-DEPTH) repeats a stage a fixed number of times at compile time. After a measurement, IF/ELSE blocks choose which later gate runs. Oracle algorithms combine all of the above.

Starter circuits: *Grover search (2 qubits)*, *T state transition across cycles*, *Recursive H (TCO expanded)* (one gate with a counting Dn badge), and *Quantum teleportation* (uses forward classical -IF conditions). In the protocol list, compile **RecursiveReversibleEchoHarness** to see recursion, measurement, and an IF/ELSE pair in one circuit.

Read: *Process composition and return values* and *Cycle-based state evolution*, *Bounded child-process recursion*, and *Structured classical branches* in the Language Reference; *Custom gates* in Examples and Troubleshooting.

Checkpoint: after an uncompute stage, what value should every temporary wire hold, and how can a truth-table test prove it? Why is INCREASECYCLE not a loop? After IF A=1 ... ELSE ... ENDIF, which gates are still in the circuit when A measures 0?

3 First circuit: from source to result

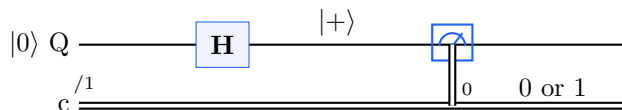
The shortest useful learning path is to prepare one qubit, transform it, measure it, and declare it as an output:

Complete runnable protocol

```
MAIN-PROCESS FirstCircuit
CREATETOKEN -I Q
SET Q 0p
H -I Q -O Q
MEASURE -I Q
RETURNVALS Q
```

3.1 Read the program line by line

1. MAIN-PROCESS FirstCircuit gives the protocol a name.
2. CREATETOKEN -I Q reserves a logical wire named Q.
3. SET Q 0p prepares Q in the basis state $|0\rangle$.
4. H -I Q -O Q applies Hadamard to that same wire, producing $|+\rangle = (|0\rangle + |1\rangle)/\sqrt{2}$.
5. MEASURE -I Q samples 0 or 1 and collapses the state.
6. RETURNVALS Q declares Q as the first public result.



3.2 What one run means

One execution gives one random-looking sample. For this circuit, theory predicts equal probability for 0 and 1. Getting 0 once does not mean H failed, and getting 1 once does not prove a bias. Repeat the circuit to compare observed frequencies with the predicted distribution.

3.3 Adding user-controlled input

Move a wire into PARAMS when callers or truth-table rows should choose its start state:

Complete runnable protocol

```
PARAMS: Input:state

MAIN-PROCESS Inverter
X -I Input -O Input
MEASURE -I Input
RETURNVALS Input
```

The matching deterministic truth table is:

Input	Output
0p	1p
1p	0p

The rest of this manual explains every symbol in these examples before moving to multi-qubit gates, child processes, and full truth-table files.

4 Classical computation foundations

Quantum circuits build on ideas from classical digital logic. Understanding bits, Boolean functions, truth tables, and reversible computation makes the QPU gate model much easier to use.

4.1 Classical bits and logical state

A classical bit has one definite logical value:

$$b \in \{0, 1\}.$$

Physical computers may represent that value with voltage, charge, magnetic orientation, or another measurable quantity. Digital logic ignores most physical detail and works with the abstraction 0 or 1.

A wire carries a bit from one operation to another. A *gate* computes a function of one or more incoming bits. For example, NOT has one input and one output:

$$\text{NOT}(0) = 1, \quad \text{NOT}(1) = 0.$$

In QPU source, a token names the logical wire. On computational-basis states, `X -I Q -O Q` has the same measured input/output relationship as classical NOT.

4.2 Boolean algebra

Boolean algebra describes functions whose values are 0 and 1. This guide uses:

Operation	Symbol	Spoken form	Meaning
NOT	$\neg A$	not A	1 when A is 0; 0 when A is 1.
AND	$A \wedge B$	A and B	1 only when both inputs are 1.
OR	$A \vee B$	A or B	1 when either or both inputs are 1.
XOR	$A \oplus B$	A exclusive-or B	1 when exactly one input is 1.
NAND	$\neg(A \wedge B)$	not-and	The complement of AND.

For bits, XOR is addition modulo two:

$$0 \oplus 0 = 0, \quad 0 \oplus 1 = 1, \quad 1 \oplus 0 = 1, \quad 1 \oplus 1 = 0.$$

There is no carry in modulo-two addition. Ordinary binary addition handles the carry separately, which is why a half-adder produces both Sum and Carry.

4.3 Truth tables as complete finite specifications

A truth table lists a function's output for every possible input pattern. With n binary inputs there are 2^n patterns. AND, OR, XOR, and NAND can be compared in one table:

A	B	$A \wedge B$	$A \vee B$	$A \oplus B$	$\neg(A \wedge B)$
0	0	0	0	0	1
0	1	0	1	1	1
1	0	0	1	1	1
1	1	1	1	0	0

For a deterministic combinational circuit, this table completely specifies the classical input/output behavior. It does not reveal the circuit’s internal gate arrangement, propagation time, power consumption, or quantum phase.

4.4 Combinational logic and stored state

A *combinational* output depends only on current inputs. Adders and the Boolean gates above are combinational. A *sequential* circuit also depends on stored state from an earlier step, as in counters and latches.

QPU cycle suffixes can document successive stages, and INCREASECYCLE marks a workspace boundary, but the compiled circuit is still one ordered gate sequence. `SAVE_STATE` and `LOAD_STATE` copy and restore the simulator state vector. They are not quantum gates. When modeling stateful behavior inside the unitary circuit, expose previous state as `PARAMS` and return next state through `RETURNVALS`.

4.5 Why ordinary classical gates can lose information

A function is reversible only if each output identifies exactly one input. Classical NOT is reversible: seeing output 1 proves the input was 0. AND is not: output 0 could have come from 00, 01, or 10.

$$(A, B) \xrightarrow{\text{AND}} A \wedge B$$

maps four input patterns onto only two output values, discarding information. Quantum evolution through closed-system gates must be unitary and therefore reversible. A quantum/reversible AND preserves both inputs and mutates another target:

$$(A, B, t) \mapsto (A, B, t \oplus (A \wedge B)).$$

Knowing the output triple lets the transformation be run backward to recover the input triple.

How to read $t' = t \oplus (A \wedge B)$

- A and B are **controls**. They are only read. After the gate they still hold exactly what they held before, which is why the input can always be recovered.
- t is the **target**, the output workspace. It is the only wire that changes. The prime in t' means “ t after the gate”.
- $\oplus$ means “flip if”. The target flips when $A \wedge B$ is 1 and stays put otherwise.
- Start the target at 0 and it ends holding $A \wedge B$, the ordinary AND answer. Start it at 1 and it ends holding NAND instead.

In the Circuit builder, A is **Control A**, B is **Control B**, and t is **Target particle**.

4.6 Controls, targets, and workspace

These terms connect classical logic to QPU syntax:

Control A wire whose value decides whether a target operation occurs. In `CNOT -I A -O T`, A is the control.

Target The wire changed by an operation. T is the target in that `CNOT`.

Workspace or ancilla An extra wire introduced to hold an intermediate or output while preserving inputs. It is commonly initialized to $|0\rangle$.

Garbage Intermediate information left in workspace after the desired answer has been computed. Reversible algorithms often *uncompute* temporary gates to return garbage wires to zero.

Uncomputation Applying inverse operations in reverse order to clean workspace without erasing information irreversibly.

The QPU language calls logical wire names tokens. `CREATETOKEN` reserves workspace, `SET` prepares it, `-I` lists inputs/controls, and `-O` names targets.

4.7 Fan-out, copying, and the no-cloning boundary

Classical logic freely copies a bit onto several wires. If the source is a known basis bit and targets begin at zero, CNOT reproduces that basis value:

$$|x\rangle|0\rangle \xrightarrow{\text{CNOT}} |x\rangle|x\rangle, \quad x \in \{0, 1\}.$$

For a superposition $\alpha|0\rangle + \beta|1\rangle$, the same circuit produces

$$\alpha|00\rangle + \beta|11\rangle,$$

not

$$(\alpha|0\rangle + \beta|1\rangle) \otimes (\alpha|0\rangle + \beta|1\rangle).$$

The first state is generally entangled; the second would be two independent copies and contains cross terms. The no-cloning theorem forbids a universal gate that copies every unknown quantum state. QPU aliases similarly give two names to one wire; they do not create a physical copy.

4.8 Deterministic, probabilistic, and quantum models

Model	State description	Transformation	Observation
Deterministic classical	One definite bit string	Boolean function	The definite output bits
Probabilistic classical	Probabilities over bit strings	Stochastic update	A sampled string
Pure-state quantum	Complex amplitudes over basis strings	Unitary matrix	A sampled string after magnitude-squared probabilities and collapse

Negative and imaginary amplitudes are not negative or imaginary probabilities. They carry phase. Two computational paths can add constructively or cancel destructively before probabilities are calculated. This interference has no counterpart in an ordinary classical random-choice table.

4.9 Classical and QPU operation map

Classical idea	QPU operation	Important difference
NOT	X or NOT	Acts linearly on superpositions; X is unitary.
XOR into target	CNOT	Preserves the control and can entangle.
AND into target	CCNOT or AND	Preserves controls; target is XOR-mutated.
Swap variables	SWAP	Exchanges complete quantum wire states.
Random bit	H then MEASURE	H first creates coherent amplitudes, not an ordinary hidden random choice.
Read a bit	MEASURE	Sampling collapses a superposition and can disturb entanglement.
Initialize zero	SET token 0p	Compiles to state preparation/reset behavior.
Function output	RETURNVALS	Declares an interface; does not itself copy or measure a wire.

5 Worked example: reversible half-adder

A half-adder adds two one-bit numbers without a carry input. Classical theory defines two outputs:

$$\text{Sum} = A \oplus B, \quad \text{Carry} = A \wedge B.$$

The name *half-adder* means it handles half of the full-adder interface: there is no C_{in} input.

5.1 Classical specification

<i>A</i>	<i>B</i>	Carry	Sum	Binary interpretation
0	0	0	0	$0 + 0 = 00_2$
0	1	0	1	$0 + 1 = 01_2$
1	0	0	1	$1 + 0 = 01_2$
1	1	1	0	$1 + 1 = 10_2$

The Carry column is the high-order bit and Sum is the low-order bit. In the last row, decimal $1 + 1 = 2$, written 10_2 in binary.

5.2 QPU protocol

Complete runnable protocol

```

PARAMS: A:state B:state

MAIN-PROCESS HalfAdder
CREATETOKEN -I Carry Sum
SET Carry 0p
SET Sum 0p

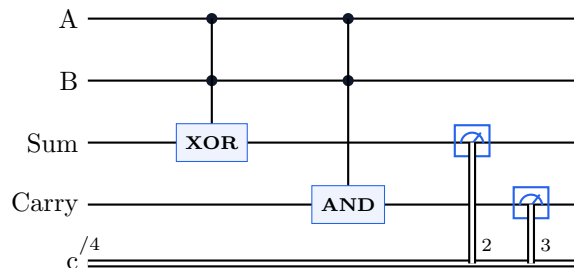
XOR -I A B -O Sum
AND -I A B -O Carry

MEASURE -I Carry
MEASURE -I Sum
RETURNVALS Carry Sum

```

5.3 What every term is doing

1. **PARAMS** declares *A* and *B* as caller-controlled state inputs. In a basis-state truth-table test they receive 0p or 1p.
2. **Carry** and **Sum** are **workspace targets**. CREATETOKEN reserves their names; SET prepares both in zero.
3. **XOR** uses *A* and *B* as **controls/inputs**. It mutates Sum according to $s' = s \oplus A \oplus B$. Because $s = 0$, the result is $A \oplus B$.
4. **AND** uses the **same preserved controls**. It mutates Carry according to $c' = c \oplus (A \wedge B)$. Because $c = 0$, Carry becomes the classical carry predicate.
5. **MEASURE** produces **classical samples**. For basis inputs this example is deterministic. If *A* or *B* is in superposition, outputs may be entangled with the inputs and measurement samples a branch.
6. **RETURNVALS** defines **interface order**. Carry is output column zero and Sum is output column one. It does not perform measurement itself.

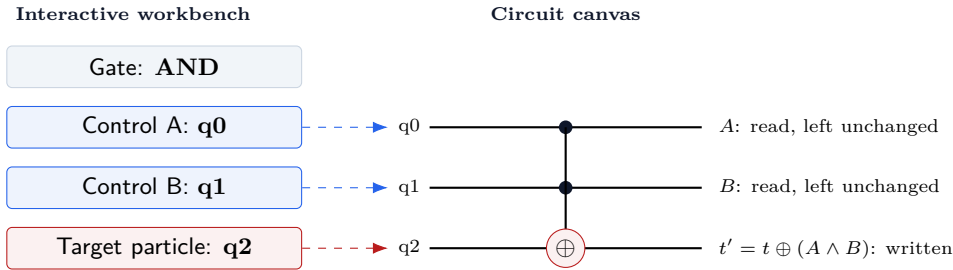


Which wires change?

Only the two outputs. A and B are *read* by both gates and come out exactly as they went in. Sum and Carry are *workspace*: they start at 0 and each receives one answer. Keeping the inputs is what makes the circuit reversible, and it is why a quantum half-adder needs four wires where a classical one draws two inputs and two outputs.

5.4 Build the Carry gate in the workbench

The AND line of the protocol corresponds to these workbench choices. With A on q0, B on q1, and Carry on q2:



Try it in the Circuit builder

Choose the *Half adder* starter circuit. Set **q0 start** and **q1 start** to each combination of 0p and 1p, choose **Run all** each time, and check the measured Sum and Carry against the table above.

5.5 Reversible state equation

Before measurement, the full basis-state action is

$$|A, B, s, c\rangle \mapsto |A, B, s \oplus A \oplus B, c \oplus (A \wedge B)\rangle.$$

The familiar half-adder is the special case $s = c = 0$. Writing the complete equation avoids a common mistake: XOR and AND do not overwrite arbitrary targets. They reversibly combine results with the targets' previous states.

5.6 Companion truth-table metadata

```
MAIN-PROCESS: HalfAdder
INPUTS: A B
OUTPUTS: Carry Sum
# A B Carry Sum
0 0p 0p 0p 0p
1 0p 1p 0p 1p
2 1p 0p 0p 1p
3 1p 1p 1p 0p
```

This file compares the measured QPU outputs with the classical half-adder specification. Passing all four rows demonstrates basis-state equivalence; it does not by itself characterize phases for superposition inputs.

6 Worked example: randomness versus interference

Classical random choice and quantum superposition can produce identical single-measurement frequencies. They are not the same theory. This example uses an intermediate phase change to distinguish them.

6.1 Classical fair coin model

Imagine a classical variable R chosen by a fair coin:

$$P(R = 0) = \frac{1}{2}, \quad P(R = 1) = \frac{1}{2}.$$

Once chosen, R is one definite value even if the observer has not looked. Applying classical NOT changes 0 to 1 and 1 to 0, but the distribution remains half and half. There is no operation that makes the probability of one hidden branch cancel the other branch.

6.2 Quantum preparation with Hadamard

Complete runnable protocol

```
MAIN-PROCESS QuantumCoin
CREATETOKEN -I Q
SET Q 0p
H -I Q -0 Q
MEASURE -I Q
RETURNVALS Q
```

Before measurement,

$$|0\rangle \xrightarrow{H} |+\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}}.$$

The amplitudes are $1/\sqrt{2}$ and $1/\sqrt{2}$; squaring magnitudes gives the same half-and-half distribution as the classical coin. If measurement happens now, this experiment alone cannot distinguish the two descriptions.

6.3 Insert phase and recombine paths

Complete runnable protocol

```
MAIN-PROCESS PhaseInterference
CREATETOKEN -I Q
SET Q 0p
H -I Q -0 Q
Z -I Q -0 Q
H -I Q -0 Q
MEASURE -I Q
RETURNVALS Q
```

Track the state:

$$|0\rangle \xrightarrow{H} \frac{|0\rangle + |1\rangle}{\sqrt{2}} \xrightarrow{Z} \frac{|0\rangle - |1\rangle}{\sqrt{2}} \xrightarrow{H} |1\rangle.$$

6.4 Term-by-term explanation

Superposition After the first H , both basis states have coherent amplitudes. No measurement has selected one.

Phase Z multiplies only the $|1\rangle$ amplitude by $-1 = e^{i\pi}$. Its magnitude remains $1/\sqrt{2}$, so immediate 0/1 probabilities remain equal.

Relative phase The plus relationship between amplitudes becomes a minus relationship. A global minus on both terms would not matter; changing only one term does.

Interference The final H mixes the basis amplitudes. Contributions to the $|0\rangle$ output cancel, while contributions to $|1\rangle$ add.

Measurement The final state is $|1\rangle$, so measurement returns 1 with probability one.

The cancellation can be seen directly from matrix multiplication:

$$H \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ -1 \end{bmatrix} = \frac{1}{2} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} 1 \\ -1 \end{bmatrix} = \frac{1}{2} \begin{bmatrix} 0 \\ 2 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}.$$

Classical probability weights are nonnegative and would add rather than cancel. Quantum amplitudes are signed or complex, so phase can create destructive and constructive interference before the Born rule turns amplitudes into probabilities.

6.5 Comparison table

Question	Classical fair coin	H–Z–H quantum circuit
State before observation	One hidden definite bit sampled fairly	Coherent complex amplitude vector
Effect of middle sign/phase	No probability analogue that preserves a fair distribution and later cancels a branch	Z changes relative phase without changing immediate basis probabilities
Can paths cancel?	No; probabilities add	Yes; amplitudes add and may cancel
Final result in this example	Still a distribution unless another classical rule forces a value	Deterministic 1

7 Every control on screen, by its exact name

This chapter lists each visible button, selector, and panel using the words printed on screen. Bold text is always an exact UI label. Use it as a map when a label is unfamiliar; hovering over most controls in the app also shows a one-line tooltip.

7.1 Navigation menu

The three-line button in the top corner (*Open site navigation*) opens the menu. Every page listed here is part of the same browser session: nothing is sent to a server for simulation.

On screen	What it does
Circuit builder	The main workbench: palette, canvas, run controls, examples, and protocol compiler.
Wiki / docs	Short in-app explanations of circuit construction, reset buttons, the learning path, and custom gates.
QPU Documentation	Opens these PDF guides inside the app.
Particle visualization	Per-qubit spheres, Bloch coordinates, the execution timeline, final state vector, and execution log.
Circuit correction lab	Tests a protocol against a truth table and helps repair it.
Upload files / Download files	Both open the file page for moving .qpucir and .qpuio files.
More	Quick links, including Reset site completely .
Reset site	Returns the whole builder to its first-visit state. See Reset buttons compared.

7.2 Documentation in the Interactive workbench

The workbench explains whatever you are examining, on the wires it uses. The card is compact until you ask for more.

On screen	What it does
Selected gate	The gate in the Gate selector, drawn on the wires Add gate to target would use. Its table columns are labelled with those wires (for example A on q0 and t on q2).
Step N of M on the canvas	Shown after Step gate , Play Sequence , or Run all : the gate that just ran, on its own wires.
Back to selected gate	Returns the card to the selectors. Changing any selector does the same.
On these wires now	The truth-table row that matches the wires' values (the start states, or the values going into the step) and what it changes, such as “q2: 0 → 1. q0 and q1 unchanged.” A wire in superposition matches several rows at once.
Compiled protocol	A second card for the compiled process: its inputs, outputs, and the row for the current start states.
Recursive frame	Shown while the step being examined is inside a recursive call. Draws the frame being run with its heading (DEPTH d · LEVEL ℓ · TCO), forward gates over a forward → rule, their inverses over a ← inverse rule, the frame's IC cycle line, and RECUR ↓ $D(d - 1)$ for the next frame. The current gate is highlighted.
Expand ... first frame	The same drawing, collapsed, whenever the circuit has a recursive call but the playhead is outside it. Click to open the first frame.
Learn more	Expands the full truth table (matching rows highlighted), how it works, the target notes, syntax, child processes, the protocol source, and the PDF link. Collapsed by default.
Try	On a row under Learn more: sets the start states to that row's inputs and rewinds the circuit.
Child process entries	Expand in place to read the child's own notes; their children expand the same way.

Long tables and lists. Anything longer than 8 items is shown in collapsible groups of at most 8, and groups are grouped again whenever there would be more than 8 of them. So 1–8 rows show directly, 9–64 use one level of groups, 65–512 two levels, and 513–4096 three. Groups of a complete truth table are named by their shared leading inputs (for example **A0 A1 = 01**). The Circuit correction lab's truth table uses the same grouping and opens the groups that contain failing rows.

7.3 Gate palette: “Pick up a block”

Click a block to select it, or drag it onto a wire of the circuit canvas. The label printed on each block is shorter than the protocol instruction name:

Block	Instruction	Block	Instruction
X, Y, Z, H, S, T	same name	CX	CNOT
P	PHASE	CCX	CCNOT (Toffoli)
CZ, CY	same name	SW	SWAP
M	MEASURE	¬	NOT
AND, NAND, OR, XOR	same name		

Preconfigured holds the built-in gates. **Custom gates** holds gates you registered from a process; it is empty until you register one.

Inverse toggles how the next drop is drawn and run. Off places the forward gate. On places its dagger: the canvas prints † on the block. S, T, and PHASE then use the negated angle. X, Y, Z, H, CNOT, CCNOT, CZ, CY, and SWAP are their own inverses, so the dagger does not change the matrix. MEASURE and the derived gates do not take the toggle. The same spelling in source is **dg** or **inv**, as in **Hdg** or **invH**.

Wrappers holds three tools that tag a gate already on the canvas instead of placing a new one:

On screen	What it does
REC	Recursive wrapper. Pick it, then click a gate: the wrapper dialog opens with Recursive wrapper ticked so you can set its depth.
IF	Pick it, then click a gate: the dialog opens with IF / ELSE wrapper ticked, Kind set to IF, and Equals set to 1.
ELSE	The same, with Kind set to ELSE and Equals set to 0.

While a wrapper tool is selected the tip under the canvas says so. Clicking the same wrapper again cancels it. Picking an ordinary gate block also cancels it.

7.4 Wrapper dialog: “Gate wrappers”

The dialog opens from a wrapper tool, or by clicking any gate that already carries a wrapper. Its heading names the gate and its target wire.

On screen	What it does
Recursive wrapper	Tags the gate as a recursive call.
Depth	The DEPTH shown in the teal badge above the gate (at least 1).
IF / ELSE wrapper	Makes the gate conditional.
Kind	IF or ELSE . Choosing ELSE sets Equals to 0; choosing IF sets it to 1. Either can then be changed. This is a display label only when Joined is ticked; the test itself comes from Equals/Value there.
Joined (test a gate)	Off (default): condition on one measured bit, using Classical bit and Equals below. On: condition on a gate expression, using the fields in <i>Joined fields</i> below instead.
Classical bit	<i>Joined off</i> . Which measured wire the condition reads, shown as $qn \rightarrow c$. That wire needs a MEASURE (M) before this gate.
Equals	<i>Joined off</i> . The value that makes the gate run: 1 or 0.
Save	Applies the ticked wrappers and rewinds the run.
Cancel	Closes the dialog without changes.
Delete wrappers	Removes both wrappers and keeps the gate.

Joined fields (shown when **Joined** is ticked).

On screen	What it does
Gate	Which gate to test: any built-in or registered custom gate with one result wire (the same rule as writing IF (GATE ...) = V — see Gate-expression conditions). MEASURE, RESET, and SWAP are not offered.
Input 1, Input 2, ...	One selector per -I wire the chosen gate takes; the count updates when you change Gate .
Output	The -O wire the result is read from.
Inverse (dg/inv)	Shown only for a reversible gate. Tests the gate’s dagger.
Phase angle	Shown only for PHASE, RX, RY, RZ, and CPHASE. θ in degrees, as in the workbench.
Test	equals or not equal to (= / !=).
Value	1p, 0p, or sp.

A line under these fields previews the exact test, for example **Test:** `AND(q0,q1) = 1p`, using the same wording the canvas label and the compiled protocol text use. Reopening a gate that already carries a gate-expression condition — written in the protocol text or built here — shows these fields filled in and ready to edit; Delete wrappers removes it.

What each wrapper changes

The **IF** / **ELSE** wrapper changes the run: the gate is skipped when the condition does not match. With **Joined** off it is written to the protocol editor as a **-IF** condition, such as `Z -I $Q2:0 -O $Q2:0 -IF Q0=0`; with **Joined** on it is written as an `IF (...) = V ... ENDIF` block around the gate. The **REC** wrapper only labels the gate with a depth badge. The gate still runs once, and the tag is not written to the protocol text. To repeat gates **DEPTH** times, write a child process with **REC** or **TREC** and call it with `RUNCHILD ... -DEPTH n`; see bounded recursion in the Language Reference.

7.5 Gate engine: “Register custom gates from QPU processes”

On screen	What it does
Gate id (palette label)	The name of the new block. Must start with a letter and cannot reuse a built-in gate name. Leave empty to use the process name.
Source process	Which process becomes the gate: <i>Current protocol editor</i> or any process in the catalog.
Color (optional — random if empty)	Block colour, as a CSS colour or gradient.
Register custom gate	Compiles the process and adds the block to Custom gates .
Remove	Deletes that custom gate from this browser session.

See *Custom gates* in Examples and Troubleshooting for how inputs and outputs map onto wires.

7.6 Circuit canvas: “Standard circuit diagram”

Each qubit row is one wire, labelled **q0**, **q1**, ... with its start state. Time runs left to right: every gate occupies its own column. When the circuit measures, one classical lane **c** sits underneath. It is a clock for those readings, not a wire the particle moves onto. A slash and a number give its width, which is the number of qubit wires. A circuit with no measurement has no **c** lane.

- **Placing a gate:** drop a block on a qubit row, or select a block and click the row. The row you drop on becomes the *target*. Controls are chosen automatically from the lowest other qubit rows; use the workbench to choose them yourself. The **c** lane cannot be targeted.
- **Removing a gate:** click the gate symbol. Clicking the meter removes that measurement step. Later gates on the same qubit stay. A gate that carries a **REC** or **IF/ELSE** wrapper opens the wrapper dialog instead; use **Delete wrappers** there first.
- **Wire labels:** after compiling a protocol, each wire also shows its parameter name (for example **A** under **q0**).
- **Line styles:** every qubit wire is a single line. **c** is the classical lane. A black double stroke, not an arrow, drops from the meter to **c** at that column. The number beside the stroke is the bit whose reading happened at that time.
- **IF / ELSE:** a conditioned gate stays on its own wire. A yellow double line joins it to **c**, and a label under **c** names the branch and its test (**IF A=1, ELSE A=0**). After the bit is measured the branch that runs shows ✓ **taken**; the other shows ⊗ **skipped** and fades with dashed lines. Skipped gates are never removed. A gate-expression **IF**, such as `IF (AND -I A B -O B) = 1p`, has no bit on **c**: its yellow line spans the wires it reads, with a yellow tap on each, and its label shows the expression (`AND(A,B) above = 1p`) using the 0p/1p/sp state vocabulary, not bare 0/1/S. The wrapper dialog’s **Joined** fields edit the same test, whether it came from the protocol text or was built there; see *Wrapper dialog*.
- **Recursive call:** drawn once, not once per frame. A teal **Dn** badge above it shows the current **DEPTH** and counts down as you step, then hides when the call is finished. A multi-gate recursive child is one **REC** box spanning its wires, captioned with the child name and depth range (for example **D5 → D1**).
- **Dashed markers:** a dashed vertical line across all wires is a whole-circuit marker, not a gate. **ICn** is an **INCREASECYCLE** boundary into cycle *n*. **S** and **L** are **SAVE_STATE** and **LOAD_STATE**, with the checkpoint name above the letter. The active marker turns red.
- **Outlines:** the step that just ran is red on a forward gate. An inverse gate is blue. The active inverse step is purple, including its connecting line. A collapsed **REC** box is red while a forward gate of the frame runs and purple while an inverse one runs.

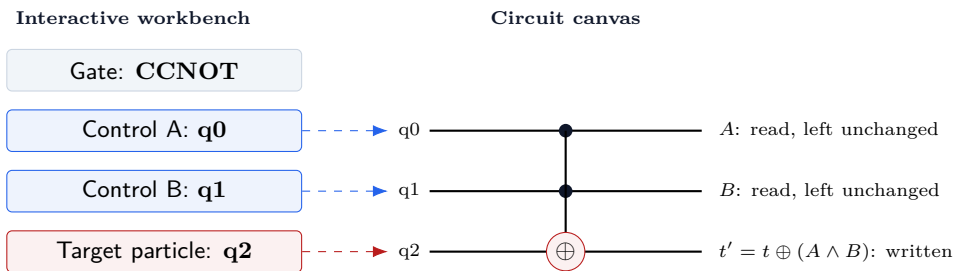
- **Dagger:** an inverse gate prints $\dagger$ on its symbol (H becomes $H^\dagger$). The forward gate keeps the plain symbol.

7.7 Interactive workbench: “Add particles, gates, and measurements”

The workbench places a gate with exactly the wires you choose.

On screen	What it does
Gate	Which gate Add gate to target will place. Same list as the palette.
Target particle	The wire the gate <i>changes</i> . For a measurement, the wire Measure target reads.
Control A	The first wire the gate <i>reads</i> . Used by CNOT, CZ, CY, and as the first input of CCNOT, AND, NAND, OR, and XOR.
Control B	The second wire the gate reads. Used by two-input gates (CCNOT, AND, NAND, OR, XOR) and custom gates with two or more inputs. SWAP also uses it as the wire to exchange with the target.
Phase angle	Appears for PHASE, RX, RY, RZ, and CPHASE. Sets θ in degrees.
Add gate to target	Appends the selected gate as the next column.
Add cycle boundary	Inserts an INCREASECYCLE marker. Advances the logical stage; does not loop or repeat earlier gates.
Add particle / Remove particle	Adds or removes the last wire (up to six on the canvas). After a compiled protocol, adds or removes a PARAMS input instead; compile again afterwards.
Measure target	Measures only the selected target wire now, without adding a meter to the circuit.
name start	One selector per input wire: 0p ($ 0\rangle$), 1p ($ 1\rangle$), or sp ($ +\rangle$). Changing it resets the run.

Gates that do not use a selector ignore it. Single-qubit gates use only **Target particle**. A control can never equal the target, so the app disables that choice and picks a free wire if needed.



Reading the picture

The selectors on the left produce the diagram on the right. **Control A** and **Control B** become the two filled dots: they are read and never changed. **Target particle** becomes the $\oplus$: it is the only wire that can change, and it changes by $t' = t \oplus (A \wedge B)$. The same mapping applies to AND. For NAND, OR, and XOR only the rule written on the target changes.

7.8 Run controls

On screen	What it does
Play Sequence / Pause sequence	Runs one gate at a time at the chosen speed, so you can watch each step. At the end it replays from the start.
Run all	Jumps straight to the final state. Useful once you trust the circuit.
Step gate	Runs exactly one more gate. Disabled at the end of the circuit.
Reset state	Rewinds the run to the start states and clears measurements. Keeps every gate.
Measure all	Measures every unmeasured wire now.
Clear circuit	Deletes every gate. Keeps the wires and their start states.
Reset site	Returns the entire builder to its default state.
Speed	Playback speed for Play Sequence .

The difference between the three reset-like buttons, and between them and the compiler's internal RESET operation, is explained in Reset buttons compared.

7.9 Examples: “Load a starter circuit”

Each card loads a small circuit onto the canvas, replacing the current gates. Cards are labelled with their step in the learning path, so they can be worked through in order.

7.10 QPU protocol compiler: “Compile text into a circuit”

On screen	What it does
SingleBitFullAdder, TwoBitFullAdder, FourBitFullAdder, PhaseDemo	Load and compile a bundled protocol in one click.
Protocol editor	The text box. Editing it does not change the canvas until you compile.
Compile protocol	Turns the editor text into canvas gates and resets the run. The sentence beside the buttons summarizes the result or the error.
Download as .qpucir / Download as -qpucir.txt	Saves the editor text. The .txt form is for devices that cannot open custom extensions.
Protocol checks	Live error and warning list for the editor text, with line numbers and suggested fixes.
Open Correction Lab for guided help	Appears when checks find a problem.
Supported operations and token map	Expands to show every accepted instruction and which wire each token name uses.

Canvas edits rewrite the editor

Placing, removing, or clearing gates on the canvas regenerates the protocol editor text from the canvas. Download a hand-written protocol before editing its compiled circuit visually.

7.11 Particle visualization

On screen	What it does
Qubit states and execution progress	One card per displayed wire: a sphere while unmeasured, the measured bit afterwards, plus its ket, r, θ, ϕ , and Bloch x, y, z . Δ values show the change caused by the last step.
Circuit execution timeline	One tile per gate, with its -I and -O wires.
Final state vector	Every nonzero amplitude with its basis label.
Outcome truth table / probabilities	Expands to list every basis outcome with its amplitude and probability.
Execution log	A sentence for every step, compile, and measurement.

7.12 File upload and download

On screen	What it does
Upload files	Choose a <code>.qpucir</code> file, optionally together with its matching <code>.qpuio</code> truth table, in one dialog. The protocol compiles immediately.
Download <i>process</i>	Saves a bundled protocol exactly as shipped.
Download current editor protocol	Saves whatever is in the protocol editor, even if it has not compiled.

7.13 Circuit correction lab

On screen	What it does
Upload protocol / Upload truth table	Load a <code>.qpucir</code> and, optionally, a <code>.qpuio</code> table.
Catalog process	Pick any compiled, uploaded, or bundled process.
Infer truth table	Build an empty table whose columns match <code>PARAMS</code> and <code>RETURNVALS</code> .
Full-adder table	Load the canonical one-bit full-adder expectations.
Probe outputs	Fill the output columns with what the current circuit actually produces. Refused for protected bundled tables.
Input columns / Output columns	Resize the table.
Test circuit	Compare the circuit with every table row; failing rows are highlighted.
Test & correct automatically	Test, then propose gate-level fixes for failing rows.
Download JSON / <code>.qpuio</code> / <code>-qpuio.txt</code> / <code>.qpucir</code> / <code>-qpucir.txt</code>	Save the table or protocol.
AI model settings	Choose the browser model or an Ollama server for the chat.
Use AI for unrecognized messages	Lets the model interpret requests the built-in command patterns do not recognize.
Send correction	Sends the chat message, for example “test the circuit”.

8 Reset: an operation that is not a gate

The word *reset* appears in two unrelated places: the compiler’s internal `RESET` operation, and three buttons in the Circuit builder. Neither is an ordinary quantum gate.

8.1 Reversible gates versus RESET

Every gate in the palette except the meter is *reversible*: its effect can always be undone. X, Y, Z, H, CNOT, CCNOT, CZ, CY, SWAP, NOT, AND, NAND, OR, and XOR undo themselves when applied a second time. S, T, and PHASE are undone by the same phase with the opposite sign. Mathematically each is a *unitary* matrix U , and $U^\dagger$ undoes it. RESET is different. It forces one wire to $|0\rangle$ whatever it held before:

$$|0\rangle \mapsto |0\rangle, \quad |1\rangle \mapsto |0\rangle, \quad \alpha|0\rangle + \beta|1\rangle \mapsto |0\rangle.$$

Two different inputs, $|0\rangle$ and $|1\rangle$, give the same output, so nothing can tell afterwards which one was there. The information has been erased, and no matrix U can undo that. RESET is therefore not unitary, just like MEASURE.

	Reversible gates (X, H, MEASURE CNOT, AND, ...)		RESET
Unitary?	Yes	No	No
Can be undone?	Yes, by the inverse gate	No	No
Result depends on chance?	No	Yes	No
Written in source?	Yes	Yes	No — write SET wire 0p
Shown on canvas?	Yes. Inverse copies add †, a blue outline, and purple when active.	Yes, as the meter on the qubit, with a double stroke down to c	No (hidden)

8.2 Where RESET comes from

You never type RESET. The compiler inserts it when a wire must start at zero:

- SET Out 0p on a workspace wire. (SET Out 1p and SET Out sp are instead prepared by applying X or H to a wire that starts at 0.)
- INCREASECYCLE, which zeroes pending workspace before the next stage.
- RUNCHILD and CALL, which zero each parent output wire once before expanding the child.

RESET steps are hidden from the canvas and filtered from the **Execution log** so that the visible circuit shows only the gates you wrote. They still run during simulation.

Reset a wire only when it is not entangled

In this simulator, RESET keeps the part of the state in which the wire is already 0 and rescales it; only if that part is empty does it move the $|1\rangle$ amplitudes to $|0\rangle$. On a fresh wire this is exactly “prepare $|0\rangle$ ”. On a wire entangled with others it also changes the other wires. For example, resetting one half of a Bell pair forces the other half to 0 as well. In multi-stage circuits, *uncompute* temporary wires back to 0 with reversible gates instead of relying on a reset.

8.3 Reset buttons compared

None of these buttons inserts a RESET operation into your circuit. They only change what the page is showing and simulating.

Kept or cleared?	Reset state	Clear circuit	Reset site	Notes
Gates on the canvas	kept	cleared	cleared	
Number of wires	kept	kept	back to 3	
Start states (<i>name</i> start)	kept	kept	all 0p	
Protocol editor text	kept	rewritten	default	Clear circuit writes an empty canvas protocol into the editor.
Compiled PARAMS / RETURNVALS	kept	cleared	cleared	
Quantum state and measurements	rewound	rewound	rewound	Back to the start states; log restarts.
Workbench selections	kept	kept	defaults	Gate H, target q0, controls q1 and q2, phase 90°.
Custom gates and catalog	kept	kept	kept	Nothing removes these except Remove or closing the browser session.

Which one do I want?

Reset state “Run the same experiment again.” Use it between repeated runs of a circuit that measures, to collect several samples.

Clear circuit “Keep my wires, start a new circuit.”

Reset site “Put everything back the way it was when I first opened the page.” The **Reset site** entry in the navigation menu and **Reset site completely** on the **More** page do the same thing.

9 Where to continue

- Read **QPU Theory Guide** when amplitude, phase, or matrix notation is unfamiliar.
- Open **QPU Language Reference** while writing exact syntax.
- Open **QPU Examples and Troubleshooting** for full adders, comparators, parity, phase kickback, small oracle circuits, and custom gates, or when Protocol checks reports an error or a truth-table test fails.